Tirumalasetti. Trinadh
192372169.

① Algorithm efficiency & Asymmetric Notation;-

Algorithm.
A step by step Procedure to solve a Problem.

↓

Input size (n).
Amount of data given to algorithm.

↓

Time complexity.
How the running time grows as input ↑.

↓

Asymmetric Notation.
Describes the growth rate for large values of n.

↓

Big-O (O)
represents an asymetric upper bound.

Big-Theta (θ)
Represents an Asymmetrically tight bound.

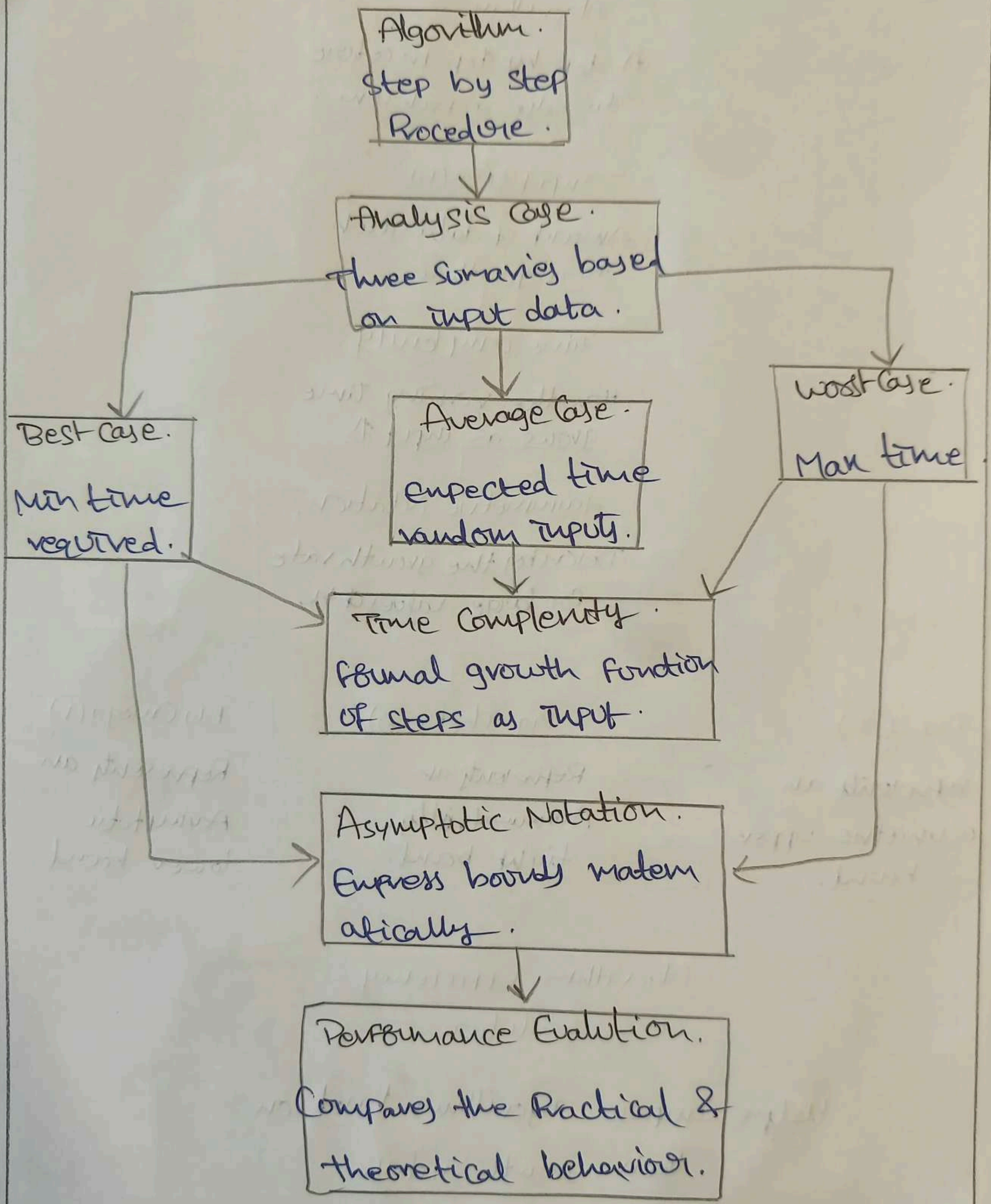
Big Omega (Ω)
Represents an Asymptotic lower bound.

↓

Algorithm Efficiency Evalution.

Helps Compare algorithms based on their growth rates.

② Performance Evalution & Analysis Case.



Tirumalasetti. Trinadh
192372169.

③ Analysing Recursive algorithms.

Divide & Conquer.

An algorithmic Rodigm that recursively a Problem into subproblems

↓

Recursive Relation.

Amathematical equation à function in terms, value on inputs.

↓

$$T(n) = aT(n/b) + F(n).$$

where a is sub-Rodem; a/b is sub Problm size, & F(n) is divide / combine Cost.

↓

Master theorem:

A code method to solve recurrency lay comparing F(n).

Case-1

Complexity

$T(n) = \Theta(n \log_b^a)$

Case-2

Complexity

$T(n) = \Theta(n \log_b^a \times \log n)$

Case-3.

Complexity

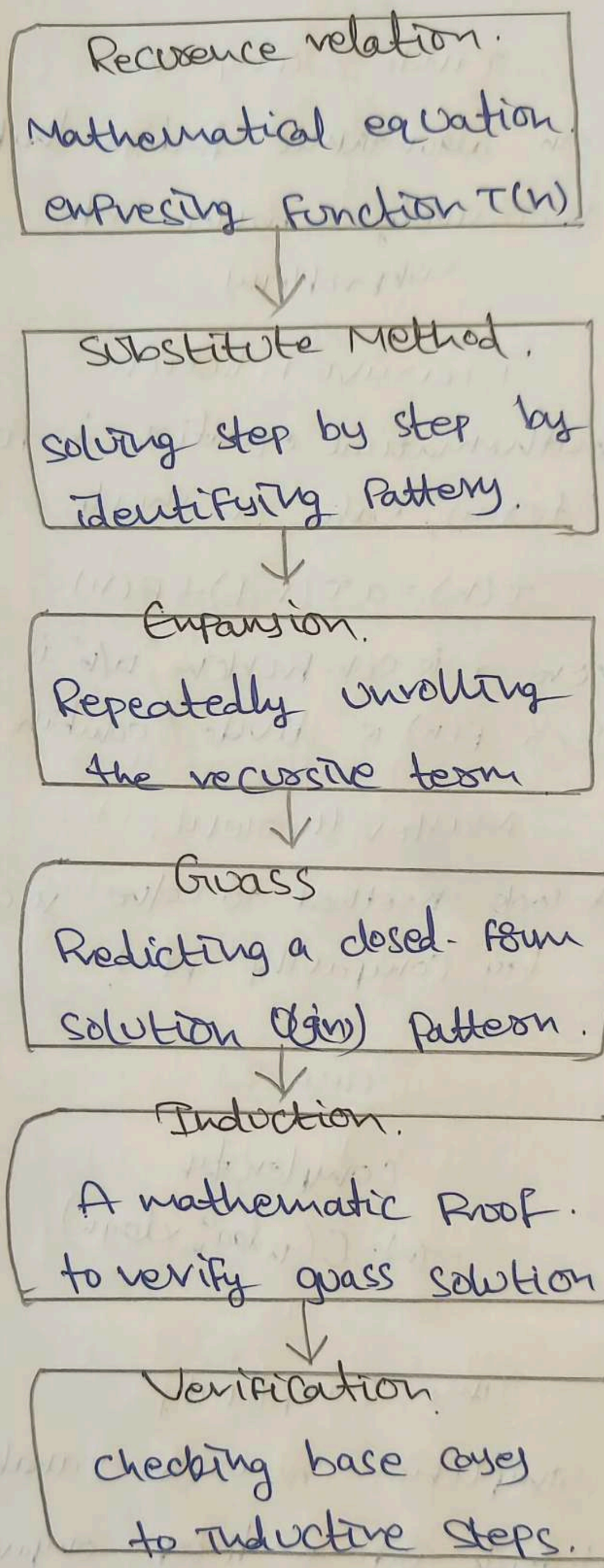
$T(n) = \Theta(F(n))$.

↓

Time Complexity.

simplifies recursive analysis by Students into a direct comparison of T(n).

① Substitution Method :- Solving recurrence relations.



(5) Complexity analysis:- Recursion Tree Method.

Recurrence Relation.

A mathematical equation describing a recursive Function T(n).

- **Branching factor (b)**
  No of child nodes Per level
- **Recursion Tree.**
  A visual tree represent recursive calls & costs.
- **level cost.**
  The sum of non-recursive work Performed.
- **Height of Tree.**
  The max Path length from root to leaf.
- **Total Cost.**
  The sum of all level costs
  Root work + level 1 + level2 + level3 --- leaves.

⑥ Tracing Performance : From operations to Big-O bound.

Algorithm.

A step-by-step Procedure to solve a Problem.

↓

Operation Count.

Tallying the basic operations executed

↓

Growth Function

A mathematical formula f(n) representing execution growth.

worst-Case Analysis.

Evaluating the max Possible operations for any INPUT Size.

↓

Big-O Notation.

An ASymptotic Notation to express the rate of growth

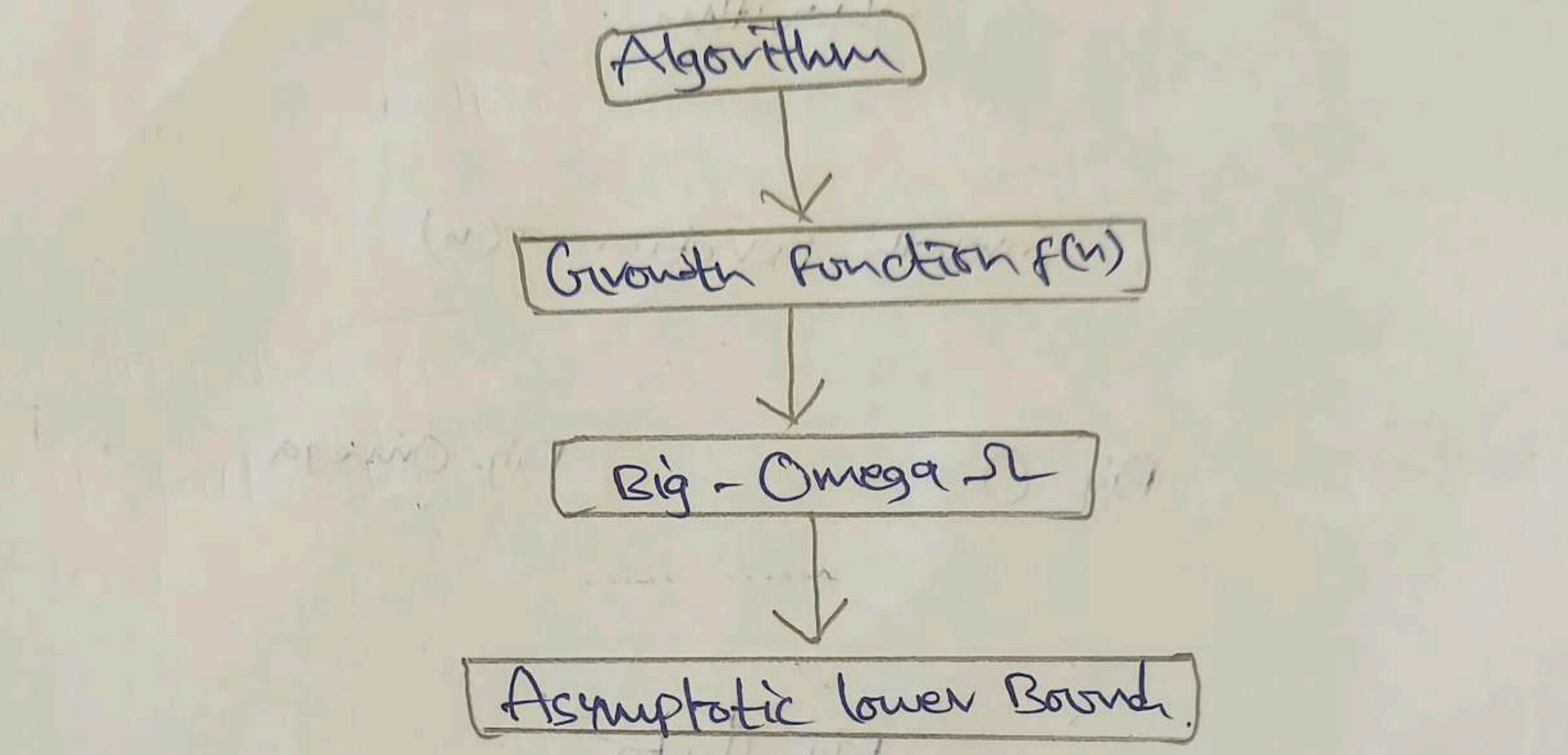
↓

Upper bound.

Runtime will not exceed this limit as n grows.

T. Trinadh
192372169.

(7) Big - Omega Notation & lower Bound.



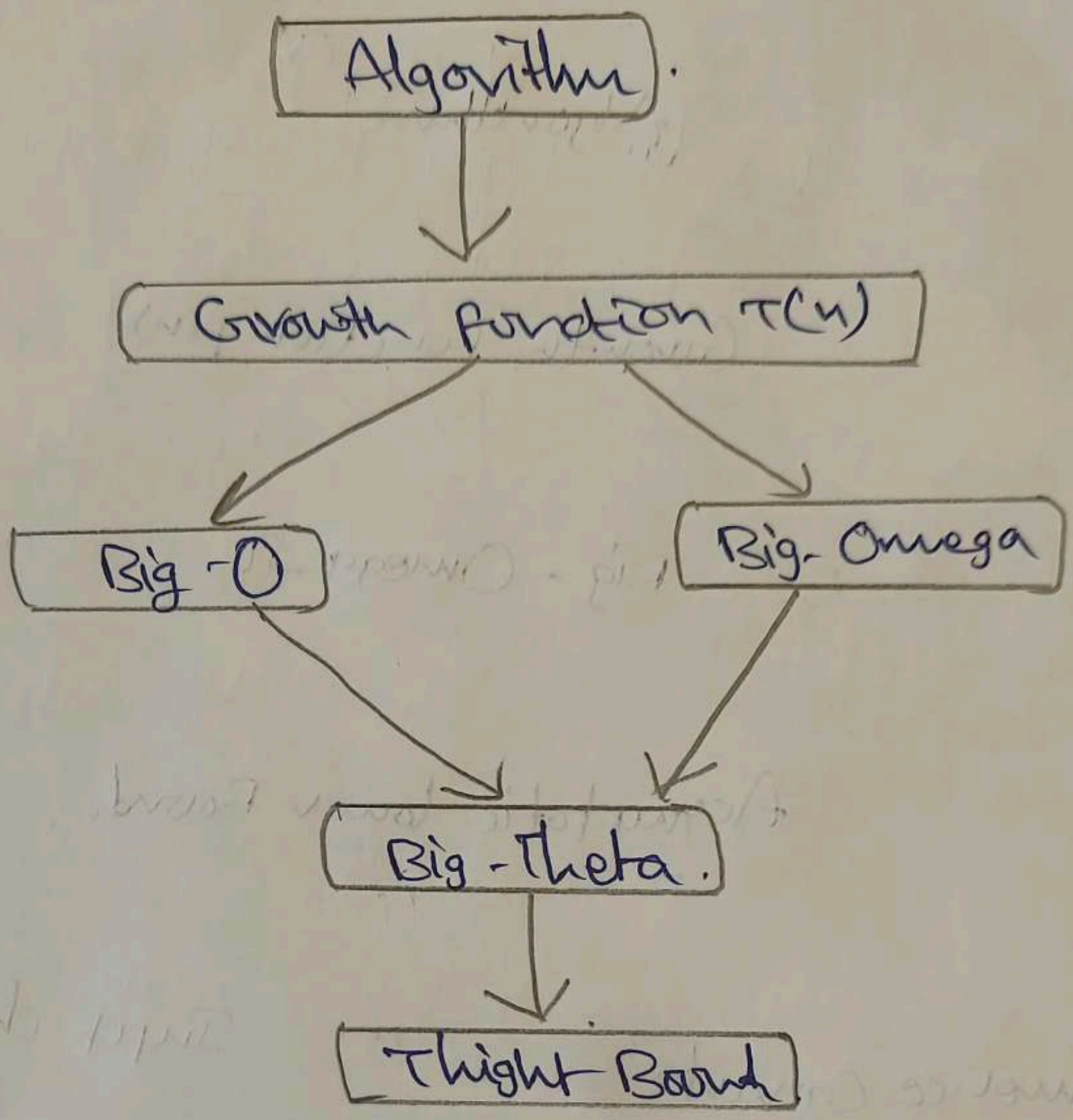
Explanation :-

* Big - Omega describes an asymptotic lower bound on a function.

* Best - Case analysis measures the minimum work over input of a given size.

* Input characteristics effect how quickly an algorithm can finish.

Big. Theta Notation & Tight Bound.



Explanation :-

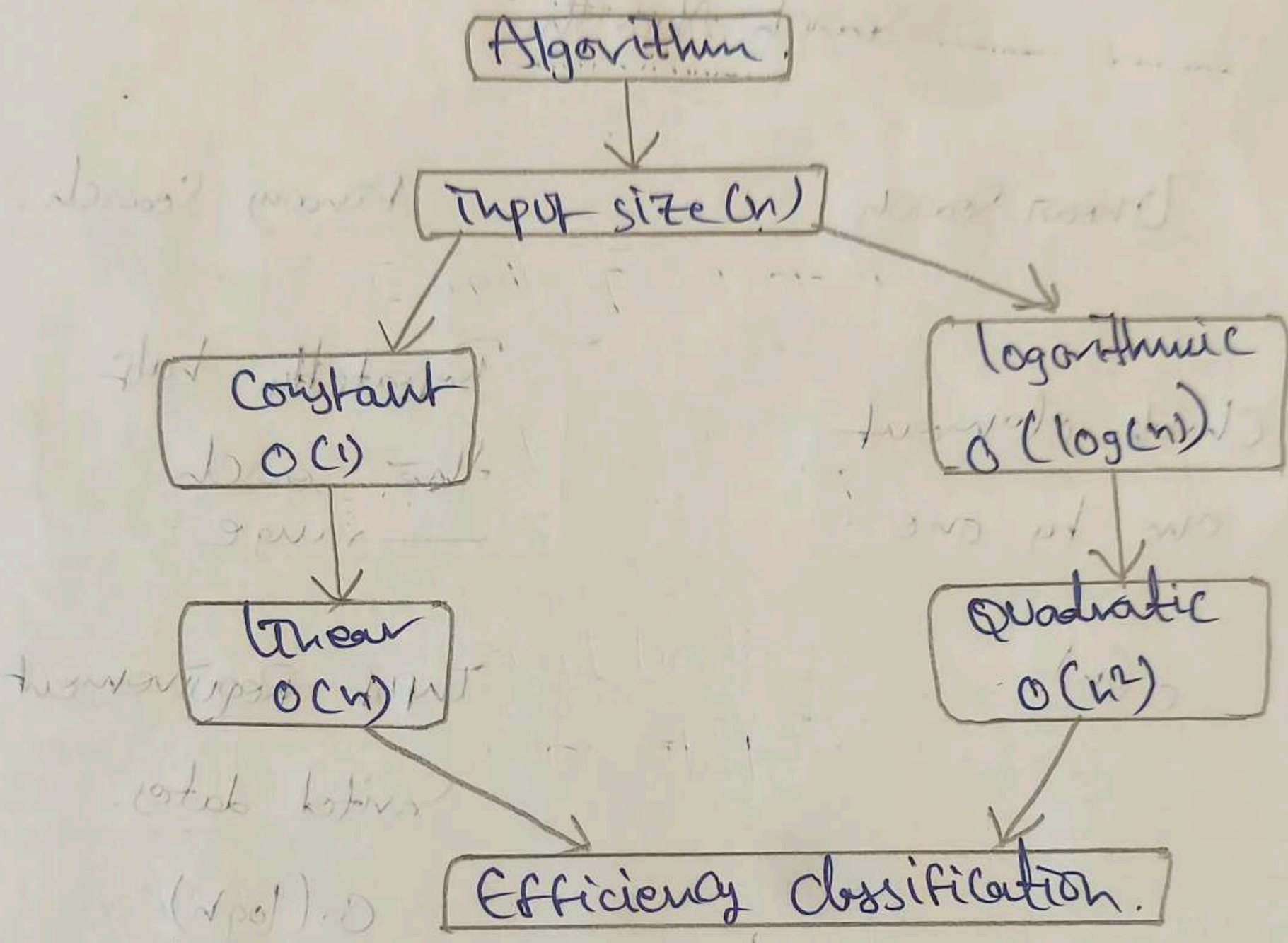
* Big-O gives an upper bound, while Big-O omega gives a lower bound.

* when both bounds match asymptotically Big-Theta describes the tight order of growth.

* $T(n) = 4n + 4$ is $O(n)$

T. Trinadh
192372169.

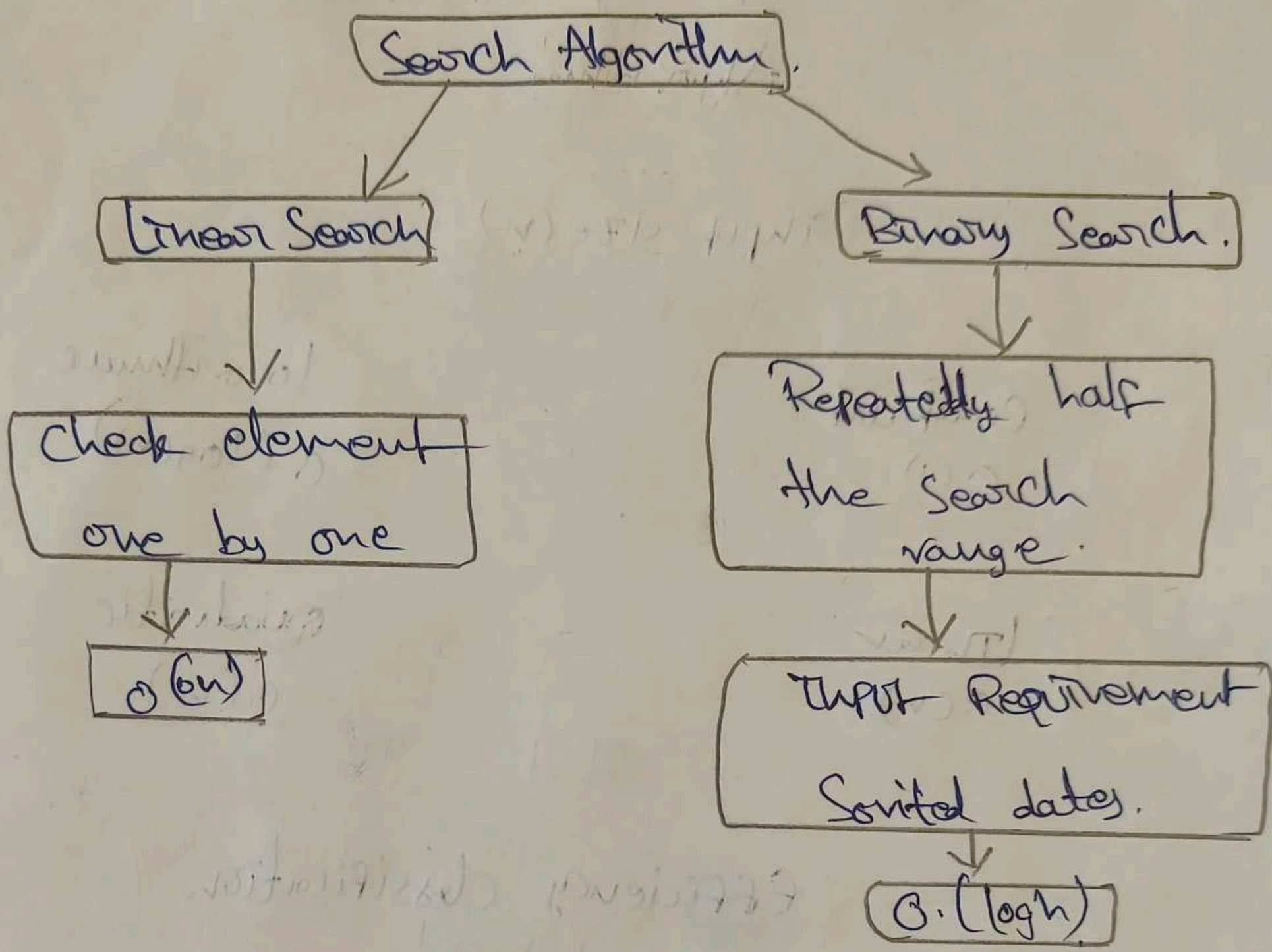
(9) Input Size & Complexity classification.



Explanation

Complexity	Growth Pattern	Example.
Constant	Does not grows with n	Array access
Logarithmic	Grows slowly.	Binary Search
Linear	Proportional to n	Linear Search
Quadratic.	Proportional to n^2.	Nested loops.

Linear Search Vs Binary Search.



Explanation:

* Linear Search examines elements one by one & may inspect all n elements.

* Binary search compares the target with middle element & eliminates the remaining search range each step.

* For logc input O(log n) than O(n)

Ex:- n = 1,024.